

ISIT307 - WEB SERVER PROGRAMMING

LECTURE 1.2 – FUNCTIONS AND CONTROL STRUCTURES



LECTURE PLAN

- Functions in PHP
- Variable scope
- Control structures
- Loop structures
- `include` and `require` statements

DEFINING FUNCTIONS

- **Functions** are groups of statements that can be executed as a single unit
 - reusable blocks of code that perform a specific task
- The syntax for defining a function is:

```
<?php  
function nameOfFunction(parameters) {  
    statements;  
}  
?>
```

DEFINING FUNCTIONS

- A **parameter** is a variable declared within the function definition
- List of parameters is placed within the parentheses following the function name
 - functions do not have to contain parameters
- The function statements are placed within the curly braces
- A **return statement** returns a value to the statement that called the function
- **Procedure** is a function that does not include a return statement

CALLING FUNCTIONS

- Function, once defined, can be used/called multiple times in any part of our code
- Function can be called using its unique name and passing the necessary arguments

EXAMPLE

```
function averageNumbers($a, $b, $c) {  
    $SumOfNumbers = $a + $b + $c;  
    $Result = $SumOfNumbers / 3;  
    return $Result;  
}  
echo averageNumbers(5,6,7);  
echo averageNumbers(5,5,7); //what is the output?
```

RETURNING VALUES

- The function can **return multiple values** within an array

```
function multiCalc($n1, $n2, $n3)
{
    $sum = $n1+$n2+$n3;    //$res[] = $n1+$n2+$n3;
    $prod = $n1*$n2*$n3;   //$res[] = $n1*$n2*$n3;
    return array($sum, $prod); //return $res;
}

$result = multiCalc(5, 6, 7);
echo "Results are: ", $result[0], " and ", $result[1];
```

PASSING ARGUMENTS TO A FUNCTION

- You can pass arguments to a function by **value** or by **reference**
 - Passing argument by value, creates a local copy of the value for the function to use
 - Passing argument by reference, passes a reference to the original variable (memory address), so the function can modify the original value of the variable (prefix the function parameter with &)

PASSING ARGUMENTS TO A FUNCTION - EXAMPLE

```
<?php
function IncrementByValue($CountByValue) {
    ++$CountByValue;
    echo "<p>IncrementByValue() value is $CountByValue.</p>"; };

function IncrementByReference(&$CountByReference) {
    ++$CountByReference;
    echo "<p>IncrementByReference() value is $CountByReference. </p>";};

$Count = 1;
echo "<p>Main program starting value is $Count.</p>";

IncrementByValue($Count);
echo "<p>Main program after call for IncrementByValue, count value is$Count. </p>";

IncrementByReference($Count);
echo "<p>Main program after call for IncrementByReference, count value is $Count.
</p>";
?>
```

PASSING ARGUMENTS TO A FUNCTION – EXAMPLE OUTPUT

functions

Main program starting value is 1.

IncrementByValue() value is 2.

Main program after call for IncrementByValue, count value is 1.

IncrementByReference() value is 2.

Main program after call for IncrementByReference, count value is 2.

FUNCTION – EXAMPLE

WHAT IS THE OUTPUT?

```
<?php
function addSome(&$text) {
    $text = $text."problem?";
}

$myText = "Is there ";
echo "<p>",$myText, "</p>";
addSome($myText);
echo "<p>",$myText, "</p>";

?>
```

VARIADIC FUNCTIONS

- PHP supports variable-length parameter lists (variable argument) - to define a function (variadic function) that accepts an arbitrary number of arguments
- Meaning you can pass 0, 1 or n number of arguments in function
- Parameter lists include `...` token preceding the last (or only) parameter in the function definition
 - passed arguments will be converted into an array

VARIADIC FUNCTIONS EXAMPLE

```
<?php
function sum(...$numbers) {
    $acc = 0;
    foreach ($numbers as $n) {
        echo $n, "<br \>";
        $acc += $n;
    }
    return $acc;
}
```

```
$sum = sum(1, 10, 23);
echo "sum = $sum";

?>
```

```
1
10
23
sum = 34
```

FUNCTIONS - OPTIONAL PARAMETERS/DEFAULT VALUES

- PHP supports optional parameters
- A function may define default values for parameters
- The default value is used only when the parameter is not specified
 - If null value is passed the default value is omitted
- Optional parameters should be specified after any required parameters

FUNCTIONS - OPTIONAL PARAMETERS/DEFAULT VALUES

```
<?php
function makeCoffee($type = "cappuccino") {
    return "<p>Making a cup of $type. </p>";
};

echo makeCoffee();
echo makeCoffee(null);
echo makeCoffee("espresso");

?>
```

FUNCTIONS – OPTIONAL DATA TYPE DECLARATIONS

- Optional data type declarations (data type hints) were introduced in PHP7+
- With this we can restrict the type of information passed into and out of a function
 - the specific data type can precede the parameter in the function definition
 - can specify the data type that a function returns
- The `declare(strict_types=1)` force values to be of the exact type expected

FUNCTIONS – OPTIONAL DATA TYPE DECLARATIONS

```
<?php
function doubleNum(int $number) : int {
    return $number *= 2;
}
$num = 5;
echo '$num =', $num, '<br>';
echo 'double_num returns ', doubleNum($num), '<br>';

$num = 4.8;
echo '$num =', $num, '<br>'; //Deprecated: Implicit conversion
                             //from float 4.8 to int loses precision -
                             //warning will be displayed, but will return 8
echo 'doubleNum returns ', doubleNum($num), '<br>';
?>
```

*If declare(strict_types=1) is used the error will be -> Fatal error: Uncaught TypeError: doubleNum():Argument #1 (\$number) must be of type int, float given
strict_types declaration must be the very first statement in the script*

FUNCTIONS - UNION TYPES

- PHP 8 now permits union types
- Union types allow to combine two or more data types for function parameters or function return value

```
<?php
declare(strict_types=1);
function doubleNum(int|float $number) : int|float {
    return $number * 2;
}

$num = 4.8;
echo '$num =', $num, '<br>';
echo 'doubleNum returns ', doubleNum($num), '<br>';
?>
```

FUNCTIONS – NAMED ARGUMENTS

- Named Arguments (introduced in PHP 8) support function call by setting the parameters by their name
- With Named Arguments the order of the arguments is not important, as long as all the required parameters are passed
 - useful to skip over multiple optional parameters

FUNCTIONS – NAMED ARGUMENTS

```
<?php
function makeSentence($name, $activity="no activity", $hours="") {
    return "Hi $name, you have $activity for $hours hrs";
}

echo makeSentence("John"), '<br>';
echo makeSentence("John", "swimming"), '<br>';
echo makeSentence("John", "swimming", "1"), '<br>';

echo makeSentence("John", activity: "hiking"), '<br>';

echo makeSentence(activity: "hiking", name: "John", hours: "8"), '<br>';
echo makeSentence("John", hours: "8", activity: "hiking"), '<br>';

echo makeSentence("John", hours: "5"), '<br>';
echo makeSentence(hours: "5", name: "John"), '<br>';

?>
```

FUNCTIONS – ANONYMOUS FUNCTIONS

- Anonymous functions do not have a specified name
- These functions are used for simple single-use operations/tasks that will not be reused elsewhere

FUNCTIONS – ARROW FUNCTIONS

- Arrow functions are a shorter syntax for anonymous functions
- The arrow function is defined with the **fn** keyword
- Syntax

`fn(parameters) => function body;`

- These functions automatically capture variables from the parent scope

FUNCTIONS – EXAMPLES

```
<?php
    $increase10 = function($p1) {
        return $p1 + 10; };

    echo $increase10(10);

    echo "<br>-----<br>";

    $count = 5;
    $multi = fn($num) => $num * $count;
    echo $multi(10);
?>
```

VARIABLE SCOPE

- **Variable scope** is where in the script the variable can be accessed and used
- A variable's scope can be either global or local
- A **global variable** is one that is declared outside a function and can be used anywhere in the script
- A **local variable** is declared inside a function and is only accessible within the function in which it is declared

THE GLOBAL KEYWORD

- You must declare a global variable with the `global` keyword inside a function definition to make the variable available within the scope of that function

```
<?php
    $glVar = "this is my value";

    function scopeExample() {
        global $glVar;
        echo "<p>$GlobalVariable</p>";
        $glVar = "if I change it";
    }
    scopeExample();
    echo "<p>$glVar </p>";
?>
```

IF STATEMENT

- **Decision making** or **flow control** is the process of determining the order in which the statements are executed
- The `if` statement is used to execute a block of code if a specific condition is true
- The syntax for a simple `if` statement is:

```
if (conditional expression)  
    statement;
```

IF/ELSE STATEMENT

- The `else` statement is used to execute a block of code if the condition within `if` is false
- The syntax for an `if...else` statement is:

if (*conditional expression*)

statement;

else

statement;

IF/ELSE STATEMENT - EXAMPLES

```
$exampleVar = 5;
if ($exampleVar == 5) {    // condition evaluates to 'TRUE'
    echo " <p>The condition evaluates to true.</p> ";
    echo '<p>$exampleVar is equal to ', " $exampleVar.</p> ";
    echo " <p>Each of these lines will be printed.</p> ";
}
echo " <p>This statement always executes after the if
                                     statement.</p> ";
```

```
$today = "Tuesday";
if ($today == "Monday")
    echo " <p>Today is Monday</p> ";
else
    echo " <p>Today is not Monday</p> ";
```

TERNARY OPERATOR & NULL COALESCING OPERATOR

- The ternary operator is a concise way of writing `if/else` statement
- The Null coalescing operator (`??`) checks whether a variable is null
 - if it is null, returns the value of the operand on the right
 - otherwise it returns the value of the variable

TERNARY OPERATOR & NULL COALESCING OPERATOR - EXAMPLES

```
$today = "Tuesday";  
echo $today == "Monday" ? "<p>Today is Monday</p>" : "<p>Today is  
not Monday</p>";
```

```
$tomorrow = $tomorrow ?? "not defined";  
echo $tomorrow;
```

SWITCH STATEMENT

- The switch statement is used when one of many blocks of code needs to be executed, based on the resulting evaluation of the expression
- The syntax for the `switch` statement is:

```
switch (expression) {  
    case label:  
        statement(s);  
        break;  
    case label:  
        statement(s);  
        break;  
    ...  
    default:  
        statement(s);  
        break;  
}
```

MATCH EXPRESSION

- `match` expression is a concise version of the switch,
 - Brings advantages in readability and functionality

```
match (expression) {  
    label => statement,  
    label => statement,  
    ...  
    default => statement,  
};
```


LOOP STRUCTURES

- A **loop structure** is a control structure that repeatedly executes a statement or a series of statements while a specific condition is `TRUE` or until a specific condition becomes `TRUE`
- There are four types of loop statements:
 - `while` statements
 - `do...while` statements
 - `for` statements
 - `foreach` statements

WHILE

- The syntax for the `while` statement is:

```
while (conditional expression) {  
    statement (s) ;  
}
```

- As long as the conditional expression evaluates to `TRUE`, the statement or block of statements executes repeatedly

WHILE – WHAT IS THE OUTPUT?

(ex. 1)

```
$count = 1;
while ($count <= 5) {
    echo "$count <br /> ";
    ++$count;
}
echo "<p>You have printed 5 numbers.</p> ";
```

WHILE - WHAT IS THE OUTPUT?

(ex. 2)

```
$count = 10;
while ($count > 0) {
    echo "$count <br /> ";
    --$count;
}
echo "<p>We have liftoff. </p> ";
-----
```

(ex. 3)

```
$count = 1;
while ($count <= 100) {
    echo "$count <br /> ";
    $count *= 2;
}
```

WHILE – INFINITE LOOP

- In an **infinite loop**, a loop statement never ends because its conditional expression is never FALSE

```
$count = 1;  
while ($count <= 10) {  
    echo "The number is $count";  
}
```

DO . . . WHILE

- Test the condition after executing a series of statements and repeats the execution as long as a given conditional expression evaluates to TRUE
- The syntax for the `do . . . while` statement is:

```
do {  
    statement(s);  
} while (conditional expression);
```

DO . . . WHILE

- `do . . . while` **statements** always execute once, before a **conditional expression** is evaluated

```
$count = 2;  
do {  
    echo "<p>The count is equal to $count</p>";  
    ++$count;  
} while ($count < 2);
```

FOR

- Combine the initialize, conditional expression, and update portions of a loop into a single statement
- It is used when we know exactly how many times the block of code to be executed

- The syntax of the `for` is

```
for (initialization; condition; update statement) {  
    statement(s);  
}
```


FOR - EXAMPLE

```
for ($count = 0; $count < 5; ++$count) {  
    echo $count, "<br />";  
}
```

FOREACH

- Used to loop through the elements in an array
- The syntax for the `foreach` statement is:

```
foreach ($array_name as $variable_name) {  
    statements;  
}
```

```
foreach ($array_name as $index_name => $variable_name)  
{  
    statements;  
}
```

FOREACH - EXAMPLE

```
$daysOfWeek = array("Monday", "Tuesday",  
                    "Wednesday", "Thursday", "Friday",  
                    "Saturday", "Sunday");
```

```
foreach ($daysOfWeek as $day) {  
    echo "<p>$Day</p>";  
}
```

```
foreach ($daysOfWeek as $dayNum => $day) {  
    echo "<p>$dayNum is $day</p>";  
}
```

INCLUDING FILES

- The `include` and `require` statements can be used to reuse the content of a file
 - These statements take all the text/code that exists in the specified file and copy it into the file using the statement
- The `include` statement generates a warning if the include file cannot be found, but the script will continue to run
- The `require` statement will throw an error and stop the execution of the script
- The `include_once` and `require_once` statements only include or require the specified file only if it hasn't been included before

INCLUDING FILES

- The file path can be either absolute or relative to the current document

```
require 'C:/wamp64/www/my_folder/script.php';
```

```
include('my_folder/script.php');  
// include 'my_folder/script.php';
```

- When using a relative file path, it's recommended to use `./` to indicate that the path begins in the current folder

```
include('./my_folder/script.php');
```

